

BSc (Hons) in **Computer Science****SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing**Practical 01**

Objective & Lecture Mapping: Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation**Task 0 : Setting up and getting the starter Code**

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" branch of the repo.

Task 1.1: The Environment State (`_init_`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

Performance Measure
Environment
Actuators
Sensors



2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

```
self.width  
self.height  
self.agent_pos  
self.walls  
self.food_positions  
self.opponents
```

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

The baseline environment is Multi-Agent because the `VisualGridHuntGame` class is designed to include both the main agent (`self.agent_pos`) and one or more opponent agents stored in `self.opponents`. During the game, each opponent moves independently using random actions in the `execute_action()` method and can interact with the main agent through collisions, which affect the score and can end the game. Although the current main function sets `num_opponents=0`, meaning no opponents are created in that specific run, the environment itself is designed to support multiple interacting agents, making it a Multi-Agent environment.

Task 1.2: The Perception Subsystem (`get_percept`)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete history of all the percepts (inputs) an agent has received from its environment. The agent uses this entire percept sequence to choose its next action.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

The `get_percept()` method represents the Percepts (P) component of the PEAS framework. It provides the information the agent can observe from the environment, such as its position, opponent positions, food detection, wall detection, collision status, current score, and remaining food. The agent uses these percepts to decide its next action.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

This environment is Fully Observable because the `get_percept()` method gives the agent all the key information about the current state, including its position, opponent positions, food detection, collision status, score, and remaining food. Since no important information is hidden from the agent, the environment is fully observable.

Task 1.3: Action Execution (`execute_action`)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

When `execute_action()` deducts points for hitting a wall, it is updating the Performance Measure (P) component of the PEAS framework. This is because the agent's score is reduced, which measures how well the agent is performing in the environment.

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is important because the Performance Measure should evaluate the agent based on its actions and their effects on the environment, not on its internal processing. In this code, the score decreases when the agent hits a wall, encouraging successful navigation and discouraging poor actions. This ensures the agent is judged by its behavior and outcomes rather than how it makes decisions.

Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (`__init__`)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment changes from Fully Observable to Partially Observable because the agent can no longer access all relevant information about the environment. Since the toxic trap locations are hidden from the `get_percept()` sensors, the agent must make decisions without knowing where the traps are, creating uncertainty in the environment state.

Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos)`) in

`self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'` , you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding `smells_toxin` helps the agent act more rationally because it provides information about possible danger in its current location. The agent can use this percept to avoid toxic traps, reduce negative outcomes, and choose actions that maximize its expected score and survival.

Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action` , check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid` , iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit was when an agent was rewarded only for keeping the environment clean, causing it to exploit the metric by removing dirt and then dumping it back to clean again repeatedly to gain more rewards. This showed that a poorly designed performance measure can encourage unwanted behavior instead of achieving the actual goal.

Finalize and Lock Lab 01 Analytical Evaluation

